

Auth Service — Documentation technique

Aucune librairie d'authentification, de rate limiting, de session ou de PoW n'est utilisée. Tout est implémenté à la main avec le module natif crypto de Node.js.

Table des matières

1. Vue d'ensemble
2. Architecture du projet
3. JWT maison (HMAC-SHA256)
4. Hachage de mot de passe (scrypt)
5. Double Pepper (microservice secondaire)
6. Rotation de token (refresh token)
7. Rate Limiter basé sur le token
8. Proof of Work (PoW)
9. Authentification inter-microservices
10. Schéma de base de données
11. Endpoints API
12. Installation et lancement
13. Variables d'environnement
14. Exemples d'utilisation (cURL)
15. Mot de passe oublié (forgot / reset)
16. Récapitulatif sécurité

1. Vue d'ensemble

Le **Auth Service** est un microservice d'authentification **zéro librairie auth** :

Ce qu'on fait	Ce qu'on n'utilise PAS
JWT signé à la main (HMAC-SHA256)	jsonwebtoken
Rate limiter maison (par token)	express-rate-limit
PoW maison (challenge HMAC)	toute lib PoW
Gestion de session maison	toute lib de session

Stack :

- Express.js
- PostgreSQL avec driver natif pg
- crypto.scrypt pour le hachage (pas de bcrypt ou autres)
- Nodemailer pour les emails (pas de Twilio car plan gratuit limitée)
-

2. Architecture du projet

```

Auth-service/
├─ src/
│   ├─ config/
│   │   ├─ database.js          # Pool PostgreSQL natif (pg)
│   │   └─ env.js              # Variables d'environnement
│   ├─ controllers/
│   │   └─ auth.controller.js   # Handlers HTTP
│   ├─ middlewares/
│   │   ├─ auth.js             # Vérification JWT maison
│   │   ├─ rateLimiter.js      # Rate limiter maison (par token)
│   │   ├─ pow.js              # Proof of Work maison (HMAC)
│   │   └─ serviceAuth.js      # Auth inter-microservices
│   ├─ routes/
│   │   ├─ auth.routes.js      # Routes publiques /api/v1/auth
│   │   └─ internal.routes.js  # Routes internes /internal
│   ├─ services/
│   │   ├─ auth.service.js     # Logique métier
│   │   ├─ mail.service.js     # Nodemailer
│   │   ├─ pepper.service.js   # Double pepper (local + remote)
│   │   └─ token.service.js    # JWT maison + rotation refresh tokens
│   ├─ utils/
│   │   ├─ jwt.js              # Implémentation JWT HMAC-SHA256
│   │   ├─ hash.js            # scrypt + timing-safe compare
│   │   └─ errors.js           # Classes d'erreur
│   └─ app.js                  # Config Express
├─ public/                     # Frontend de test
│   ├─ index.html              # Interface de test (navigateur)
│   └─ test.js                 # Logique client (PoW solver, appels API)
├─ init.sql                    # Schéma PostgreSQL
├─ .env
├─ package.json
└─ server.js                   # Point d'entrée

```

3. JWT maison (HMAC-SHA256)

Implémentation : src/utils/jwt.js

On a implémenté le standard JWT (RFC 7519) uniquement avec crypto natif.

Structure d'un token :

HEADER.PAYLOAD.SIGNATURE

Chaque partie est encodée en **Base64URL** (pas Base64 classique) :

- Header: { "alg": "HS256", "typ": "JWT" }
- Payload: { "sub": "user-uuid", "email": "...", "role": "user", "iat": ..., "exp": ... }
- Signature: HMAC-SHA256(header_b64 + "." + payload_b64, secret)

Création (sign)

1. Encoder le header en Base64URL
2. Ajouter iat (issued at) et exp (expiration) au payload
3. Encoder le payload en Base64URL
4. Calculer HMAC-SHA256 sur "header.payload" avec le secret
5. Encoder la signature en Base64URL
6. Retourner "header.payload.signature"

Vérification (verify)

1. Séparer le token en 3 parties
2. Recalculer HMAC-SHA256 sur "header.payload"
3. Comparer avec la signature reçue via crypto.timingSafeEqual() → empêche les attaques par timing
4. Décoder le payload, vérifier que exp > maintenant
5. Retourner le payload

Pourquoi timingSafeEqual ? Une comparaison classique (===) s'arrête au premier octet différent. Un attaquant pourrait mesurer le temps de réponse pour deviner la signature octet par octet. timingSafeEqual compare TOUS les octets en temps constant.

4. Hachage de mot de passe (scrypt)

Implémentation : src/utils/hash.js

Pourquoi pas bcrypt ?

- bcrypt est une dépendance native C++ (problèmes de compilation)
- bcrypt ne résiste pas aussi bien aux attaques GPU/ASIC
- crypto.scrypt est intégré à Node.js, zéro dépendance

Pourquoi scrypt ? scrypt est une fonction de dérivation de clé **memory-hard** : elle nécessite beaucoup de mémoire en plus du CPU. Cela rend les attaques par GPU/ASIC très coûteuses.

Paramètres :

Paramètre	Valeur	Rôle
N	16384	Coût CPU/mémoire
r	8	Taille de bloc
p	1	Parallélisme
keyLength	64	Taille de la clé dérivée

Flux de hachage

1. Générer un salt aléatoire de 32 octets
2. Concaténer le mot de passe + le double pepper
3. Appliquer `scrypt(input, salt, 64)` avec les paramètres ci-dessus
4. Stocker en DB : "salt_hex:derived_key_hex"

Flux de vérification

1. Extraire le salt et le hash depuis la valeur stockée
2. Recalculer `scrypt(password + pepper, salt, 64)`
3. Comparer avec `crypto.timingSafeEqual()` (pas de `===`)

5. Double Pepper (microservice secondaire)

Implémentation : `src/services/pepper.service.js` + dossiers `pepper-primary/` et `pepper-service/`

Concept : Le **pepper** est un secret cryptographique ajouté au mot de passe avant le hachage. Contrairement au salt (stocké en DB), le pepper est **hors de la base de données**. On utilise un **double pepper entièrement externe**, le Auth Service ne stocke aucun pepper.

Pepper	Source	Stockage
Primaire	pepper-primary (microservice)	.env du pepper-primary
Secondaire	pepper-service (microservice)	.env du pepper-service

Combinaison :

`combined_pepper = HMAC-SHA256(PRIMARY_PEPPER, SECONDARY_PEPPER)` (On utilise HMAC et pas une simple concaténation pour que le pepper combiné soit une valeur dérivée cryptographiquement forte.)

Les deux Pepper Services Chaque service suit la même architecture :

- Serveur **HTTP natif** (`http.createServer`), pas d'Express
- Protégé par X-Service-Key (timing-safe comparison)
- Endpoint unique : GET `/pepper`
- En production : réseau interne, pas d'accès public

Service	Port par défaut	Clé Auth Service
pepper-primary	3098	PEPPER_PRIMARY_KEY
pepper-service	3099	PEPPER_SECONDARY_KEY

Avantage :

Sécurité : Le Auth Service ne contient **aucun pepper**. Un attaquant doit compromettre les trois services pour reconstituer le pepper complet.

Mode développement : Si aucun PEPPER_PRIMARY_URL / PEPPER_SECONDARY_URL n'est configuré, le Auth Service utilise un fallback déterministe (uniquement en NODE_ENV=development). En production, les deux services sont obligatoires.

6. Rotation de token (refresh token)

Implémentation : src/services/token.service.js

Concept À chaque utilisation d'un refresh token, l'ancien est **marqué comme révoqué** et un nouveau est émis. Si un token révoqué est réutilisé, c'est une **attaque par vol de token** → toutes les sessions du user sont détruites.

Flux normal

1. User se connecte : reçoit Access Token (15 min) + Refresh Token T1
2. Access Token expire : User envoie T1 à POST /refresh
3. Serveur : T1 existe et n'est PAS revoked : Marquer T1 comme revoked → Générer T2 + nouvel Access Token

Détection de vol

1. Attaquant vole T1
2. Le vrai User utilise T1 : reçoit T2 (T1 marqué revoked)
3. L'Attaquant utilise T1 (déjà revoked !) : DELETE tous les refresh tokens du user → L'attaquant ET le user sont déconnectés → Le user doit se reconnecter (reset de sécurité)

Stockage Les refresh tokens sont stockés **hashés** en SHA-256 dans la table refresh_tokens.

7. Rate Limiter basé sur le token

Implémentation : src/middlewares/rateLimiter.js

Pourquoi basé sur le token et pas l'IP ?

- Derrière un proxy/VPN, des milliers d'utilisateurs partagent la même IP
- Un attaquant peut changer d'IP facilement
- Le token identifie **l'utilisateur réel**

Implémentation (maison en mémoire avec Map, sans librairie)

1. Extraire le user ID du JWT
2. Chercher/créer une entrée dans la Map : { count, resetAt }
3. Incrémenter le compteur
4. Si count > max → HTTP 429
5. Headers : X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset
6. Nettoyage périodique (setInterval 60s)

Trois niveaux

Limiter	Clé	Cible	Config
tokenLimiter	req.user.id	Routes authentifiées	100 req / 15 min
preAuthTokenLimiter	JWT decode (sub)	POST /refresh	100 req / 15 min
challengeLimiter	IP (fallback)	GET /pow-challenge	30 req / 1 min

Les routes signup/login sont protégées uniquement par le **PoW**.

8. Proof of Work (PoW)

Implémentation : src/middlewares/pow.js

Flux

1. Client : GET /api/v1/auth/pow-challenge : { challenge, difficulty, signature, expiresAt }
2. Client résout : trouver un nonce tel que SHA-256(challenge + nonce) commence par difficulty zéros hexadécimaux
3. Client : POST /signup ou /login avec headers : x-pow-challenge, x-pow-nonce, x-pow-signature

Vérification serveur

1. Vérifier la signature HMAC du challenge (timingSafeEqual)
2. Vérifier que le challenge n'est pas expiré
3. Vérifier SHA-256(challenge + nonce) commence par N zéros

Pourquoi stateless ? Le challenge est signé par HMAC avec un secret serveur + timestamp intégré : pas de stockage DB.

Difficulté :

POW_DIFFICULTY=4 → ~65 000 tentatives en moyenne (~100 ms sur navigateur moderne).

9. Authentification inter-microservices

Implémentation : src/middlewares/serviceAuth.js + src/routes/internal.routes.js

Middleware serviceAuth

1. Lire le header X-Service-Key
2. Comparer avec INTER_SERVICE_KEY via SHA-256 + timingSafeEqual
3. Si valide : req.isService = true

Route interne POST /internal/validate-token Header: X-Service-Key Body: { "accessToken": "eyJhbG..." }

Réponse : { "valid": true, "user": { "id", "email", "role" } } ou erreur.

Flux

1. User envoie token au Users Service
2. Users Service appelle Auth Service /internal/validate-token
3. Auth Service vérifie le JWT et retourne le user

10. Schéma de base de données

Table users

Colonne	Type	Description
id	UUID (PK)	gen_random_uuid()
email	VARCHAR(255)	UNIQUE NOT NULL
phone	VARCHAR(20)	Optionnel
password_hash	TEXT	scrypt (salt:hash)
first_name	VARCHAR(100)	Prénom
last_name	VARCHAR(100)	Nom
email_verified	BOOLEAN	Défaut: false
phone_verified	BOOLEAN	Défaut: false
role	VARCHAR(20)	'user' ou 'admin'
deleted	BOOLEAN	Soft delete
created_at	TIMESTAMPTZ	Date de création
updated_at	TIMESTAMPTZ	Dernière modification

Table refresh_tokens

Colonne	Type	Description
id	UUID (PK)	gen_random_uuid()
user_id	UUID (FK)	→ users.id ON DELETE CASCADE
token_hash	TEXT	SHA-256 du refresh token
revoked	BOOLEAN	Pour la détection de réutilisation
expires_at	TIMESTAMPTZ	Expiration (7 jours)
created_at	TIMESTAMPTZ	Date de création

Table email_verifications

Colonne	Type	Description
id	UUID (PK)	gen_random_uuid()
user_id	UUID (FK)	→ users.id
code	VARCHAR(6)	Code à 6 chiffres
expires_at	TIMESTAMPTZ	15 min après création
used	BOOLEAN	Défaut: false
created_at	TIMESTAMPTZ	Date de création

Table password_resets

Colonne	Type	Description
id	UUID (PK)	gen_random_uuid()
user_id	UUID (FK)	→ users.id ON DELETE CASCADE
code	VARCHAR(6)	Code à 6 chiffres
expires_at	TIMESTAMPTZ	15 min après création
used	BOOLEAN	Défaut: false
created_at	TIMESTAMPTZ	Date de création

11. Endpoints API

Routes publiques — /api/v1/auth

Méthode	Route	Protection	Description
GET	/pow-challenge	Rate limit (IP)	Obtenir un challenge PoW
POST	/signup	PoW	Inscription
POST	/login	PoW	Connexion
POST	/forgot-password	PoW	Demande de réinitialisation
POST	/reset-password	PoW	Réinitialisation du mot de passe
POST	/refresh	Rate limit (token)	Rotation des tokens
POST	/logout	JWT + Rate (token)	Déconnexion
GET	/me	JWT + Rate (token)	Profil connecté
POST	/verify-email	JWT + Rate (token)	Vérification email

Routes internes — /internal

Méthode	Route	Protection	Description
POST	/validate-token	X-Service-Key	Validation de token pour les autres microservices

12. Lancement

Ordre de démarrage

1. pepper-primary
2. pepper-service
3. Auth Service (port 3001)

13. Variables d'environnement

Auth Service (.env)

Variable	Description	Défaut
PORT	Port du serveur	3001
NODE_ENV	Environnement	development
PG_HOST	Hôte PostgreSQL	localhost
PG_PORT	Port PostgreSQL	5432
PG_DATABASE	Nom de la base	auth_service
PG_USER	Utilisateur PostgreSQL	postgres
PG_PASSWORD	Mot de passe PostgreSQL	postgres
JWT_ACCESS_SECRET	Secret HMAC pour les access tokens	<i>requis</i>

JWT_REFRESH_SECRET	Secret HMAC pour les refresh tokens	<i>requis</i>
JWT_ACCESS_EXPIRES_IN	Durée access token	15m
JWT_REFRESH_EXPIRES_IN	Durée refresh token	7d
PEPPER_PRIMARY_URL	URL du pepper-primary	<i>requis en prod</i>
PEPPER_PRIMARY_KEY	Clé partagée avec pepper-primary	<i>requis en prod</i>
PEPPER_SECONDARY_URL	URL du pepper-service	<i>requis en prod</i>
PEPPER_SECONDARY_KEY	Clé partagée avec pepper-service	<i>requis en prod</i>
EMAIL_USER	Adresse Gmail	<i>requis</i>
EMAIL_PASS	App password Gmail	<i>requis</i>
POW_DIFFICULTY	Zéros requis pour le PoW	4
POW_SECRET	Secret HMAC pour les challenges PoW	<i>requis</i>
POW_EXPIRY_SECONDS	Durée de validité d'un challenge	300
INTER_SERVICE_KEY	Clé pour l'auth inter-microservices	<i>requis</i>
RATE_LIMIT_WINDOW_MS	Fenêtre du rate limit (ms)	900000
RATE_LIMIT_MAX_REQUESTS	Max requêtes par fenêtre	100

pepper-primary (.env)

- PORT = 3098
- SERVICE_KEY = PEPPER_PRIMARY_KEY
- PEPPER_VALUE = valeur du pepper primaire

pepper-service (.env)

- PORT = 3099
- SERVICE_KEY = PEPPER_SECONDARY_KEY
- PEPPER_VALUE = valeur du pepper secondaire

14. Exemples d'utilisation (cURL)

Health check

```
curl http://localhost:3001/health
```

Obtenir un challenge PoW

```
curl http://localhost:3001/api/v1/auth/pow-challenge
```

Inscription

```
curl -X POST http://localhost:3001/api/v1/auth/signup \
  -H "Content-Type: application/json" \
  -H "x-pow-challenge: CHALLENGE" \
  -H "x-pow-nonce: NONCE" \
  -H "x-pow-signature: SIGNATURE" \
```

```
-d
'{"email":"jean@example.com","password":"motDePasse123","firstName":"Jean",
"lastName":"Dupont"}'
```

Connexion (idem avec /login et seulement email + password)

Profil

```
curl http://localhost:3001/api/v1/auth/me -H "Authorization: Bearer
ACCESS_TOKEN"
```

Rafraîchir les tokens

```
curl -X POST http://localhost:3001/api/v1/auth/refresh -H "Content-Type:
application/json" -d '{"refreshToken":"REFRESH_TOKEN"}'
```

Vérifier l'email

```
curl -X POST http://localhost:3001/api/v1/auth/verify-email \
-H "Authorization: Bearer ACCESS_TOKEN" \
-H "Content-Type: application/json" \
-d '{"code":"482619"}'
```

Déconnexion

```
curl -X POST http://localhost:3001/api/v1/auth/logout \
-H "Authorization: Bearer ACCESS_TOKEN" \
-d '{"refreshToken":"REFRESH_TOKEN"}'
```

Mot de passe oublié et Réinitialiser (identiques aux exemples signup avec PoW headers)

Validation inter-services

```
curl -X POST http://localhost:3001/internal/validate-token \
-H "X-Service-Key: votre-cle-inter-service" \
-d '{"accessToken":"ACCESS_TOKEN"}'
```

Interface de test Ouvrir <http://localhost:3001> dans le navigateur (frontend intégré qui résout le PoW automatiquement).

15. Mot de passe oublié (forgot / reset)

Implémentation : `src/services/auth.service.js` + `src/services/mail.service.js`

Flux

1. POST /forgot-password avec { email } : Toujours même réponse (anti-énumération) → Si email existe : code 6 chiffres stocké dans password_resets, envoyé par email (15 min)
2. User reçoit l'email
3. POST /reset-password avec { email, code, newPassword } : Vérifie code (non utilisé, non expiré) : Hash nouveau mot de passe (scrypt + double pepper) : Met à jour DB : Révoque TOUS les refresh tokens du user (déconnexion forcée de toutes sessions)

Sécurité

- Anti-énumération totale
- PoW obligatoire
- Code à usage unique
- Révocation totale après reset

16. Récapitulatif sécurité

Mesure	Implémentation
JWT	HMAC-SHA256 maison (crypto), timingSafeEqual
Hachage mot de passe	scrypt (memory-hard) + salt unique + double pepper
Double pepper	Local (.env) + Remote (Pepper Service), combinés via HMAC
Rotation refresh token	Revocation + détection de réutilisation (vol de token)
Rate limiting	Maison, basé sur le token (user ID), pas l'IP
Proof of Work	Challenge HMAC stateless, SHA-256 avec difficulté ajustable
Auth inter-services	X-Service-Key + SHA-256 timingSafeEqual
Anti-timing attack	crypto.timingSafeEqual partout
Injection SQL	Requêtes paramétrées exclusivement
Headers sécurité	helmet (CSP, X-Frame-Options, etc.)
Soft Delete	Champ deleted
Refresh token hashé	SHA-256 en DB, jamais en clair
Reset password	Code 6 chiffres, 15 min, usage unique, révocation totale
Anti-énumération	forgot-password ne révèle jamais si l'email existe

Librairies auth utilisées : aucune.